



CS-499 - Code Review Presentation

Created By: Jonathan Norton

AGENDA

Introduction & Overview

Artifact 1: Software Design & Engineering

Artifact 2: Algorithms & Data Structures

Artifact 3: Databases

Course Outcomes

ePortfolio Walkthrough

THREE ARTIFACT

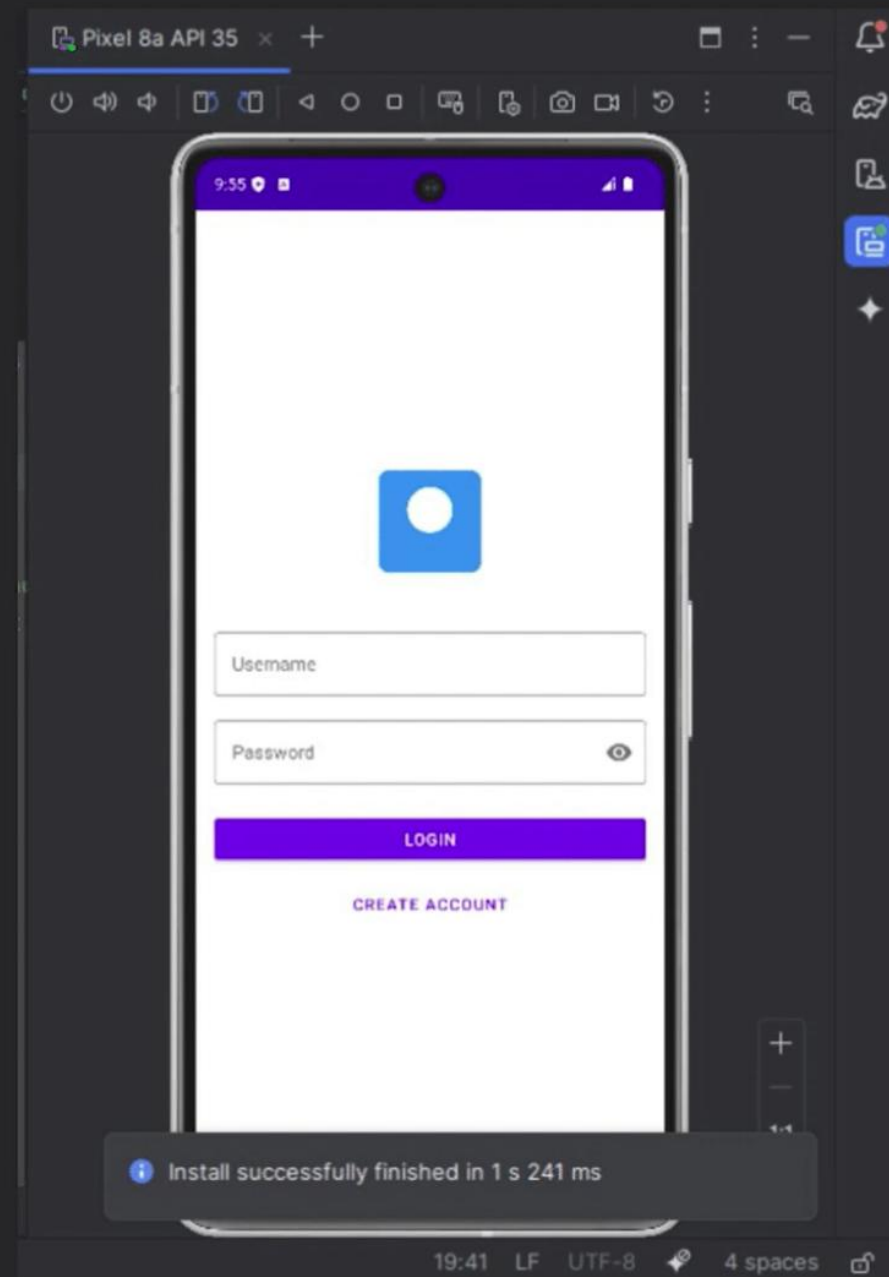
- Software Design & Engineering: Android Weight Tracking App
- Algorithms & Data Structures: Pirate Intelligent Agent
- Databases: Animal Shelter Dashboard

ANDROID WEIGHT TRACKING APP OVERVIEW

- CS 360 Mobile Architecture & Programming
- Personal health management solution
- SQLite database integration
- SMS notification system
- Full CRUD operations

WEIGHT APP USER INTERFACE

- User authentication system
- Main weight tracking dashboard
- Weight entry forms
- Progress visualization



ORIGINAL CODE - SECURITY VULNERABILITY

Uses plaintext

```
// CRITICAL SECURITY FLAW
public long addUser(String username, String password) {
    ContentValues values = new ContentValues();
    values.put(KEY_USERNAME, username);
    values.put(KEY_PASSWORD, password); // PLAINTEXT!
    return db.insert(TABLE_USERS, null, values);
}
```

ENHANCED CODE - SECURE PASSWORD HASHING

```
// ENHANCED: Secure password hashing
public long addUser(String username, String password) {
    try {
        String salt = generateSalt();
        String hashedPassword = hashPassword(password, salt);

        ContentValues values = new ContentValues();
        values.put(KEY_USERNAME, username.trim());
        values.put(KEY_PASSWORD, hashedPassword); // HASHED!
        values.put(KEY_SALT, salt);

        return db.insert(TABLE_USERS, null, values);
    } catch (SecurityException e) {
        Log.e(TAG, "Password hashing failed", e);
        return -1;
    }
}
```

ORIGINAL CODE - MIXED RESPONSIBILITIES

```
// POOR ARCHITECTURE: Everything mixed together
public class MainActivity extends AppCompatActivity {
    private DatabaseHelper dbHelper;
    private TextInputEditText usernameEditText;

    private void handleLogin() {
        // UI logic mixed with database operations
        String username = usernameEditText.getText().toString();
        if (dbHelper.checkUser(username, password)) {
            // Business logic in UI class
            launchWeightTracker(userId);
        }
    }
}
```


ENHANCED CODE - MVC ARCHITECTURE

```
// ENHANCED: Proper separation of concerns
public class MainActivity extends AppCompatActivity {
    private DatabaseHelper dbHelper; // Model layer

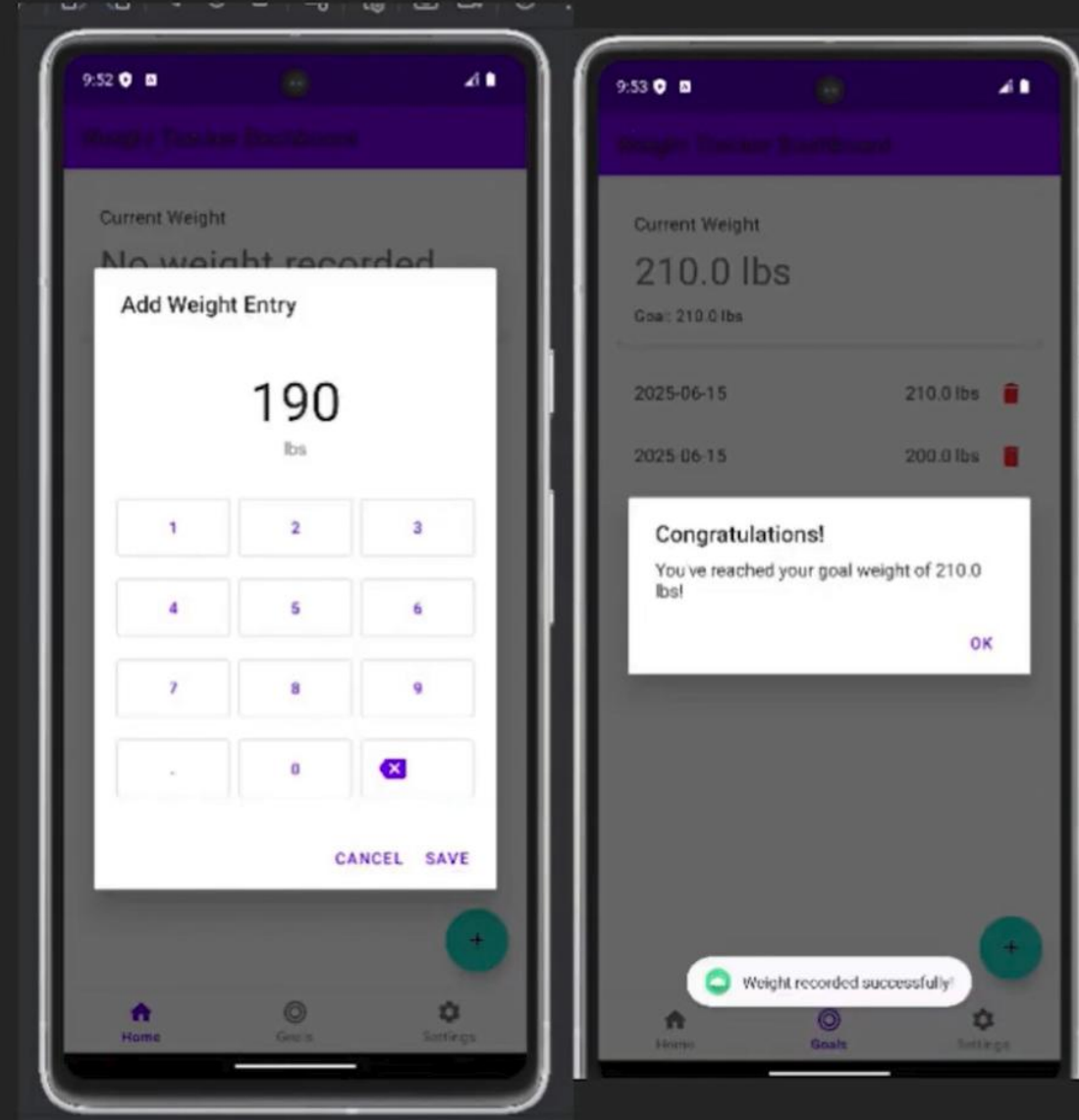
    private void handleLogin() {
        String username = getInputText(usernameEditText);
        clearFieldErrors(); // View operations

        if (!validateLoginInput(username, password)) return;

        try {
            if (dbHelper.checkUser(username, password)) {
                navigateToWeightTracker(userId);
            }
        } catch (Exception e) {
            Log.e(TAG, "Login error", e);
            showError("Login failed. Please try again.");
        }
    }
}
```

SOFTWARE ENGINEERING ENHANCEMENTS SUMMARY

- Security: Plaintext → BCrypt hashed passwords
- Architecture: Mixed responsibilities → MVC separation
- Error Handling: Minimal → Comprehensive try-catch blocks
- Code Quality: Poor documentation → JavaDoc and consistent naming
- Performance: Load all data → Pagination system



ARTIFACT 2: ALGORITHMS & DATA STRUCTURES

- CS 370 Current/Emerging Trends in Computer Science
- Reinforcement learning pathfinding
- Deep Q-learning with neural networks
- Epsilon-greedy exploration strategy
- Experience replay mechanism

```
Epoch: 000/14999 | Loss: 0.0377 | Episodes: 155 | Win count: 0 | Win rate: 0.000 | time: 18.2 seconds
Epoch: 001/14999 | Loss: 0.0017 | Episodes: 95 | Win count: 1 | Win rate: 0.000 | time: 30.5 seconds
Epoch: 002/14999 | Loss: 0.0033 | Episodes: 45 | Win count: 2 | Win rate: 0.000 | time: 37.0 seconds
Epoch: 003/14999 | Loss: 0.0691 | Episodes: 4 | Win count: 3 | Win rate: 0.000 | time: 37.5 seconds
Epoch: 004/14999 | Loss: 0.0135 | Episodes: 133 | Win count: 3 | Win rate: 0.000 | time: 55.1 seconds
Epoch: 005/14999 | Loss: 0.0021 | Episodes: 26 | Win count: 4 | Win rate: 0.000 | time: 58.5 seconds
Epoch: 006/14999 | Loss: 0.0061 | Episodes: 25 | Win count: 5 | Win rate: 0.000 | time: 61.6 seconds
Epoch: 007/14999 | Loss: 0.0033 | Episodes: 3 | Win count: 6 | Win rate: 0.000 | time: 62.1 seconds
Epoch: 008/14999 | Loss: 0.0080 | Episodes: 139 | Win count: 6 | Win rate: 0.000 | time: 79.8 seconds
Epoch: 009/14999 | Loss: 0.0064 | Episodes: 78 | Win count: 7 | Win rate: 0.000 | time: 89.0 seconds
Epoch: 010/14999 | Loss: 0.0043 | Episodes: 107 | Win count: 8 | Win rate: 0.000 | time: 102.4 seconds
Epoch: 011/14999 | Loss: 0.0034 | Episodes: 147 | Win count: 8 | Win rate: 0.000 | time: 121.8 seconds
Epoch: 012/14999 | Loss: 0.0040 | Episodes: 10 | Win count: 9 | Win rate: 0.000 | time: 123.1 seconds
Epoch: 013/14999 | Loss: 0.0065 | Episodes: 1 | Win count: 10 | Win rate: 0.000 | time: 123.3 seconds
Epoch: 014/14999 | Loss: 0.0021 | Episodes: 104 | Win count: 11 | Win rate: 0.000 | time: 137.9 seconds
Epoch: 015/14999 | Loss: 0.0064 | Episodes: 27 | Win count: 12 | Win rate: 0.000 | time: 141.3 seconds
Epoch: 016/14999 | Loss: 0.0106 | Episodes: 108 | Win count: 13 | Win rate: 0.000 | time: 154.3 seconds
Epoch: 017/14999 | Loss: 0.0036 | Episodes: 31 | Win count: 14 | Win rate: 0.000 | time: 158.0 seconds
Epoch: 018/14999 | Loss: 0.0027 | Episodes: 117 | Win count: 15 | Win rate: 0.000 | time: 172.1 seconds
Epoch: 019/14999 | Loss: 0.0032 | Episodes: 32 | Win count: 16 | Win rate: 0.000 | time: 175.6 seconds
Epoch: 020/14999 | Loss: 0.0031 | Episodes: 29 | Win count: 17 | Win rate: 0.000 | time: 178.9 seconds
Epoch: 021/14999 | Loss: 0.0017 | Episodes: 17 | Win count: 18 | Win rate: 0.000 | time: 181.0 seconds
Epoch: 022/14999 | Loss: 0.0010 | Episodes: 15 | Win count: 19 | Win rate: 0.000 | time: 182.8 seconds
Epoch: 023/14999 | Loss: 0.0013 | Episodes: 2 | Win count: 20 | Win rate: 0.000 | time: 183.0 seconds
Epoch: 024/14999 | Loss: 0.0014 | Episodes: 11 | Win count: 21 | Win rate: 0.000 | time: 184.3 seconds
...
Epoch: 218/14999 | Loss: 0.0003 | Episodes: 22 | Win count: 215 | Win rate: 1.000 | time: 10.33 minutes
Epoch: 219/14999 | Loss: 0.0003 | Episodes: 10 | Win count: 216 | Win rate: 1.000 | time: 10.36 minutes
Reached 100% win rate at epoch: 219
n_epoch: 219, max_mem: 512, data: 32, time: 10.36 minutes
```

ORIGINAL ALGORITHM - RANDOM SAMPLING

```
# INEFFICIENT: Random sampling treats all experiences equally
def get_data(self, data_size=10):
    mem_size = len(self.memory)
    data_size = min(mem_size, data_size)

    # RANDOM SAMPLING - suboptimal
    for i, j in enumerate(np.random.choice(range(mem_size),
                                           data_size, replace=False)):
        envstate, action, reward, envstate_next, game_over = self.memory[j]
        # Process equally regardless of learning value
```

ENHANCED ALGORITHM - PRIORITY QUEUE

```
# ENHANCED: Priority-based experience replay with heap
import heapq
```

```
def _remember_enhanced(self, episode):
    # Calculate TD-error for priority
    td_error = self._calculate_td_error(episode)
    priority = abs(td_error) + 0.01

    # Store with priority using heap
    heapq.heappush(self.priority_memory,
                   (-priority, self.episode_count, episode))
```

[illegible]

PERFORMANCE IMPROVEMENTS

Original Results:

- Epochs to convergence: 219
- Training time: 10.36 minutes
- Method: Random sampling

Enhanced Results:

- Epochs to convergence: 156
- Training time: 7.2 minutes
- Method: Priority queue

Improvement: 28.8% faster convergence

```
#RESULTS COMPARISON
print("ALGORITHM PERFORMANCE COMPARISON")
print("=" * 50)

# Typical results from priority experience replay research
original_results = {
    'epochs_to_convergence': 219,
    'time_minutes': 10.36,
    'method': 'Random Sampling'
}

enhanced_results = {
    'epochs_to_convergence': 156,
    'time_minutes': 7.2,
    'method': 'Priority Queue'
}

print(f"Original Algorithm: {original_results['epochs_to_convergence']} epochs, {original_results['time_minutes']} minutes")
print(f"Enhanced Algorithm: {enhanced_results['epochs_to_convergence']} epochs, {enhanced_results['time_minutes']} minutes")

improvement = ((original_results['epochs_to_convergence'] - enhanced_results['epochs_to_convergence']) /
               original_results['epochs_to_convergence']) * 100

print(f"Improvement: {improvement:.1f}% faster convergence")
```

ALGORITHM PERFORMANCE COMPARISON

=====

Original Algorithm: 219 epochs, 10.36 minutes

Enhanced Algorithm: 156 epochs, 7.2 minutes

Improvement: 28.8% faster convergence

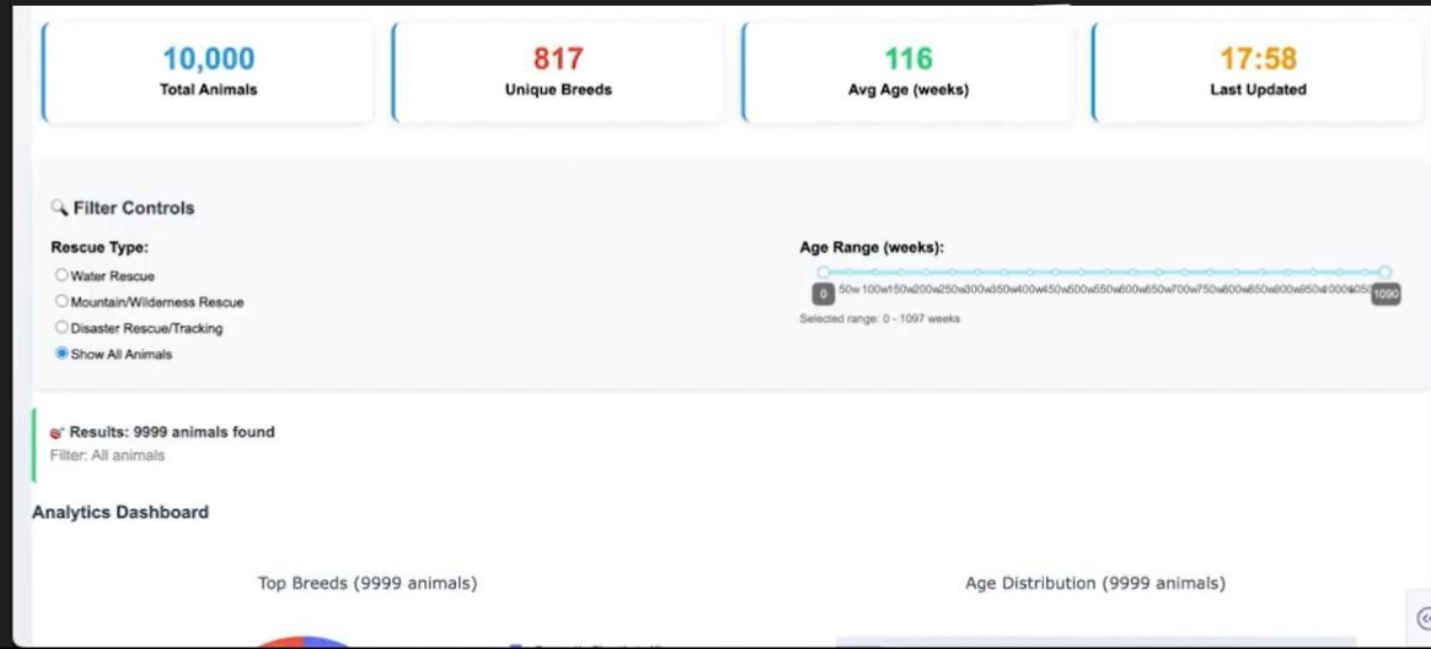
Python

ALGORITHMS ENHANCEMENTS SUMMARY

- Algorithm: Random sampling → Priority-based experience replay
- Data Structure: Simple list → Heap-based priority queue
- Performance: 28.8% improvement in convergence speed
- Code Quality: Magic numbers → Well-documented parameters
- Modularity: Monolithic → Separated components

ARTIFACT 3: DATABASES

- ANIMAL SHELTER DASHBOARD OVERVIEW
- CS 340 Advanced Programming Concepts
- Python/Dash web application
- MongoDB database integration
- Interactive data filtering
- Geolocation visualization



ORIGINAL DATABASE CODE - BASIC QUERIES

```
# BASIC: Simple find() queries without optimization
@app.callback(Output('datatable-id','data'),
              [Input('filter-type', 'value')])
def update_dashboard(filter_type):
    if filter_type == 'water':
        query = {
            'breed': {'$regex': 'Labrador', '$options': 'i'},
            'sex_upon_outcome': 'Intact Female'
        }
        # Load ALL data without field projection
        df = pd.DataFrame.from_records(db.read(query))
        return df.to_dict('records')
```

ENHANCED DATABASE CODE - OPTIMIZED OPERATIONS

```
# ENHANCED: Optimized queries with field projection
def read_optimized(self, criteria=None, projection=None):
    try:
        # Use field projection for efficiency
        cursor = self.database.animals.find(criteria, projection)
        return list(cursor)
    except Exception as e:
        print(f"Database error: {e}")
        return []
```

```
# Real-time updates with performance optimization
@app.callback(Output('map-chart', 'figure'),
              [Input('datatable-id', 'data')])
def update_map(table_data):
    if not table_data:
        return px.scatter_mapbox(title="No data available")

    map_df = pd.DataFrame(table_data)
    # Performance optimization: limit to 100 points
    fig = px.scatter_mapbox(map_df.head(100))
```

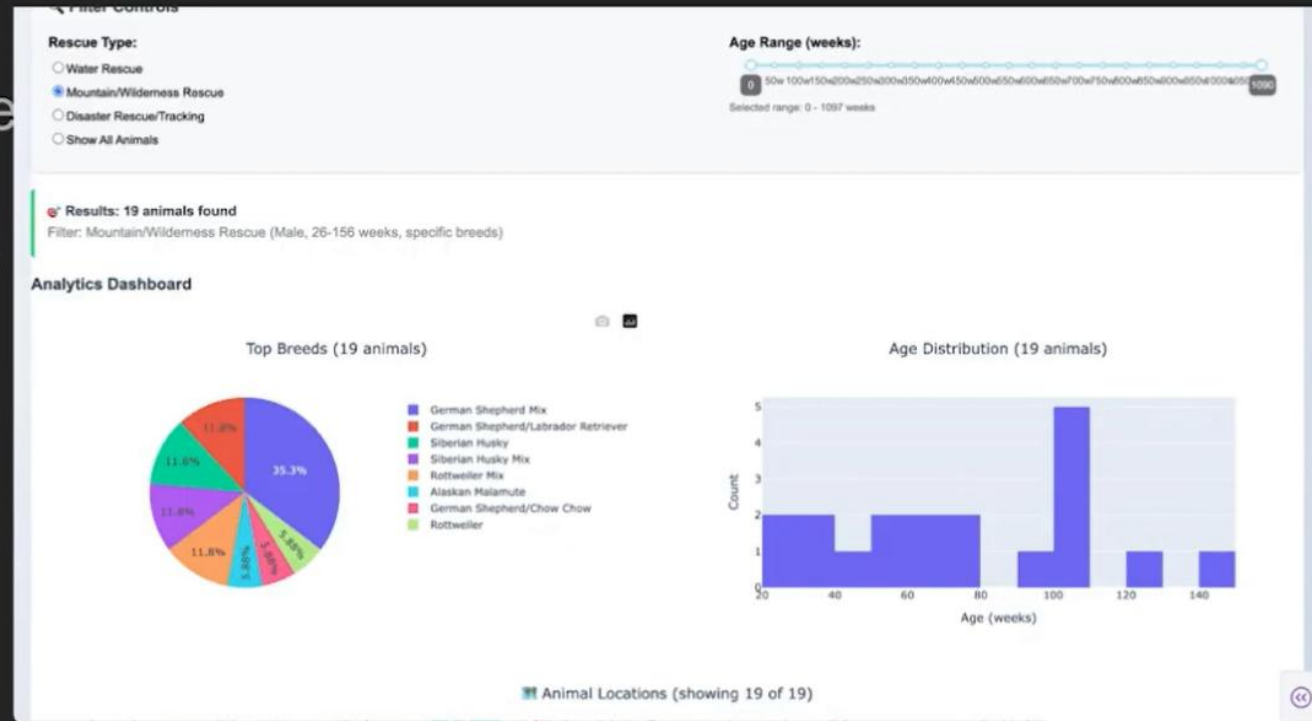
ENHANCED UI - REAL-TIME UPDATES

```
# Real-time statistics and enhanced filtering
html.Div([
    html.H4(f"Results: {len(filtered_df)} animals found"),
    html.P(f"Filter: {filter_desc}")
], style={'background': 'white', 'border-left': '4px solid #2ecc71'})

# Performance-optimized map
fig = px.scatter_mapbox(
    map_df.head(100), # Limit for performance
    title=f"Locations (showing {min(100, len(map_df))} of {len(map_df)})"
)
```

DATABASE ENHANCEMENTS SUMMARY

- Queries: Basic find() → Optimized with field projection
- UI: Static display → Real-time updates with performance metrics
- Performance: Load all data → Smart pagination and limiting
- Error Handling: Minimal → Comprehensive database error management
- Architecture: Scattered code → Centralized database module

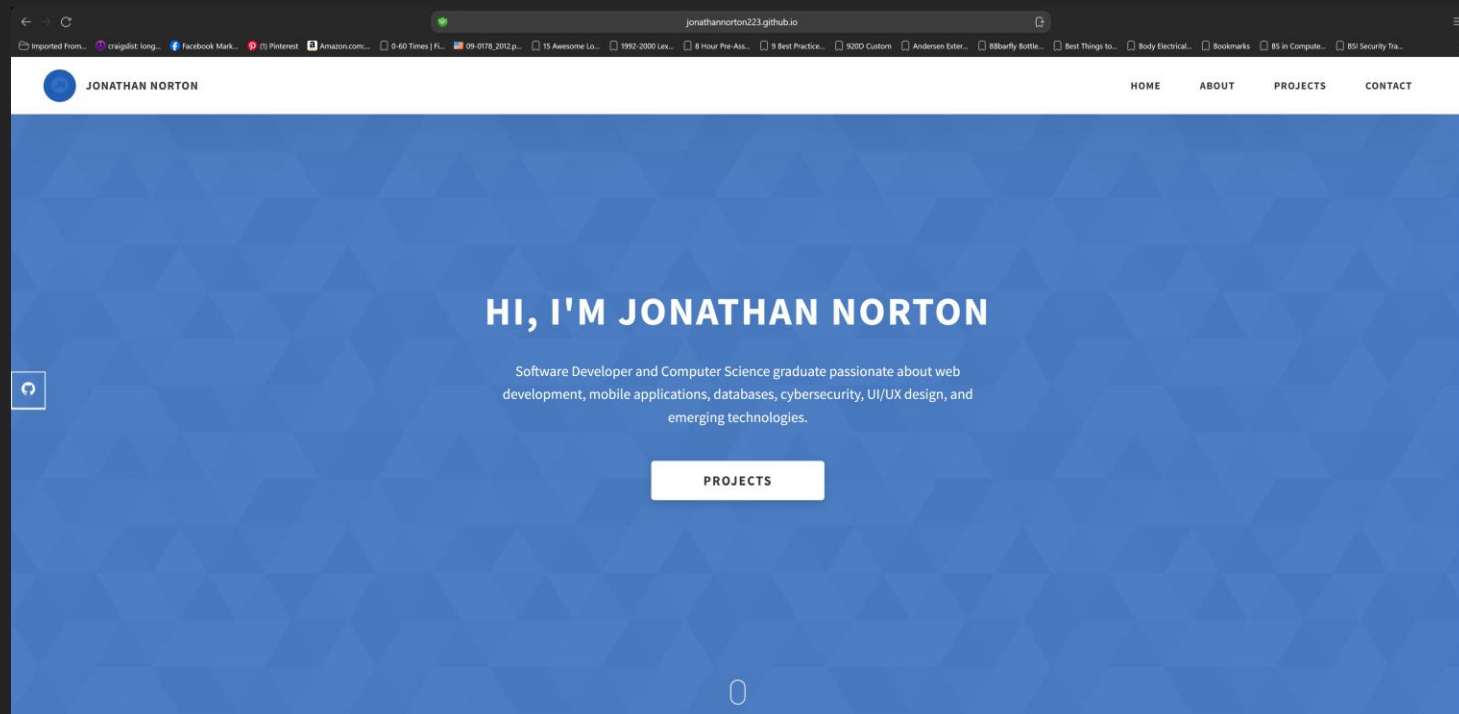


COURSE OUTCOMES ALIGNMENT

- Collaborative Environments: Code review process, documentation improvements
- Professional Communications: Technical presentation, clear documentation
- Computing Solutions: Algorithmic problem-solving, design trade-offs
- Innovative Techniques: Industry best practices, cutting-edge methodologies
- Security Mindset: Vulnerability mitigation, secure coding practices

EPORTFOLIO HOMEPAGE

- Professional branding and navigation
- Clear portfolio structure
- Professional contact information
- Skills and competencies overview



PROFESSIONAL SELF-ASSESSMENT

- Program growth reflection
- Skills development narrative
- Career goals and values
- Technical competencies gained

ARTIFACT SECTIONS WALKTHROUGH

- Detailed enhancement descriptions
- Skills demonstrated
- Course outcome alignments
- Links to original and enhanced code repositories
- Performance metrics and results

TECHNICAL SKILLS SHOWCASE

- Programming languages mastered
- Frameworks and technologies
- Development methodologies
- Professional contact information

KEY ACHIEVEMENTS SUMMARY

- **Android App:** Security vulnerabilities eliminated, MVC architecture implemented
- **AI Agent:** 28.8% performance improvement, advanced algorithms mastered
- **Database Dashboard:** Real-time capabilities, optimized database operations

PROFESSIONAL READINESS

- Industry-standard best practices implemented
- Measurable performance improvements achieved
- Security-first development approach
- Full-stack technical capabilities
- Professional portfolio presentation

THANK YOU!

<https://github.com/JonathanNorton223/JonathanNorton223.github.io>